Kernix benchmark report: 0.5.0

Measured 2026-10-20T15:26:00Z on 12th Gen Intel(R) Core(TM) i3-1215U.

Rust: rustc 1.99.0 (b940084d7 2026-09-28); NumPy: 2.5.4; one CPU thread; dtype in workload name.

Source: `ff5d1685f32f64a5d843d02cef023758f40b0d37`; working tree dirty: True.

Comparative CPU results

Workload	Kernix p50 / p95 (us)	ndarray p50 / p95 (us)	nalgebra p50 / p95 (us)	NumPy p50 / p95 (us)
add_65536_f64	28.161 / 38.342	25.570 / 31.411	not measured	25.860 / 27.785
cholesky_64_f64	27.112 / 31.757	not measured	14.475 / 21.233	29.566 / 30.742
conv_1x3x32x32_8x3x3x3_f32	13.660 / 15.052	62.102 / 71.496	not measured	119.688 / 143.845
create_full_64x64_f64	0.337 / 0.408	0.292 / 0.322	not measured	1.720 / 2.082
create_matrix_3x3_f64	0.059 / 0.060	0.019 / 0.019	not measured	1.175 / 1.341
dense_sgd_step_32x16x8_f64	12.800 / 13.065	1.125 / 1.348	not measured	not measured
matmul_128_f64	95.847 / 112.419	93.352 / 97.832	not measured	812.491 / 872.103
matmul_32_f64	1.833 / 1.906	1.686 / 1.927	not measured	12.592 / 12.937
matmul_reuse_128_f64	95.106 / 105.828	93.695 / 100.167	not measured	806.599 / 844.373
matmul_reuse_32_f64	1.671 / 1.810	1.691 / 1.829	not measured	12.696 / 15.150
qr_256x64_f64	573.412 / 582.906	not measured	451.608 / 471.080	810.888 / 869.434
qr_32x16_f64	6.710 / 9.392	not measured	4.590 / 4.704	18.985 / 24.067
qr_8x4_f64	0.623 / 0.795	not measured	0.295 / 0.310	11.224 / 13.798
qr_96x32_f64	53.289 / 59.836	not measured	40.569 / 43.967	89.623 / 101.965
rgb1080_to224_f32	573.215 / 782.186	768.796 / 971.391	not measured	4425.306 / 5875.687
softmax_256x64_f64	95.105 / 103.632	84.505 / 92.752	not measured	113.751 / 122.408
solve_64_f64	33.730 / 37.059	not measured	23.444 / 26.172	48.595 / 58.091
sum_65536_f64	6.835 / 8.183	6.299 / 6.752	not measured	11.622 / 18.175

Method and limits

Operations allocate outputs except scalar sums and explicitly named matmul_reuse cases. Reuse cases time preallocated output writes in all libraries. Inputs are preallocated and contiguous. Kernix and ndarray share the matrixmultiply GEMM engine (no external BLAS); NumPy uses the build recorded in raw.json. BLAS/OMP thread counts are fixed to one. CPU affinity: 0. Warmed sequential samples on a shared laptop; CPU frequency and other system load are uncontrolled. p95 is the maximum of 15 batch means, not a per-request tail-latency guarantee. Different sum orders may round differently. The ndarray softmax is an explicitly coded stable rowwise comparison, not a native ndarray softmax API. NumPy timings include Python call overhead. No GPU comparison. Vision workloads, when present: ndarray uses explicit bilinear/conv loops, while NumPy uses vectorized bilinear interpolation and sliding-window einsum. No native convolution/resize API is implied for ndarray. RGB input is 1920x1080 packed u8; output is 1x3x224x224 f32 (602112 bytes), with no full-resolution float tensor. No claim that a complete Full HD model reaches 120 FPS follows from these measurements. The dense SGD case compares a validated reverse-mode Kernix graph against explicit analytical ndarray derivatives, including weight reset/copy in both. Graph lifetime, finite validation and parameter safety have costs; these are different API levels. LU solve includes factorization; QR materializes thin Q and R in all measured libraries. Construction cases produce owned dynamically shaped tensors/arrays in every library; they include allocation, filling/moving values and metadata validation. No fixed-size stack-only matrix is timed against a heap-owned tensor. Python overhead dominates small arrays. Historical reports were not pinned and lack source archives; their trends are indicative. source.tar.gz preserves the exact current source; SHA-256 is recorded in raw.json. raw.json contains all measurements, dependencies and environment metadata.

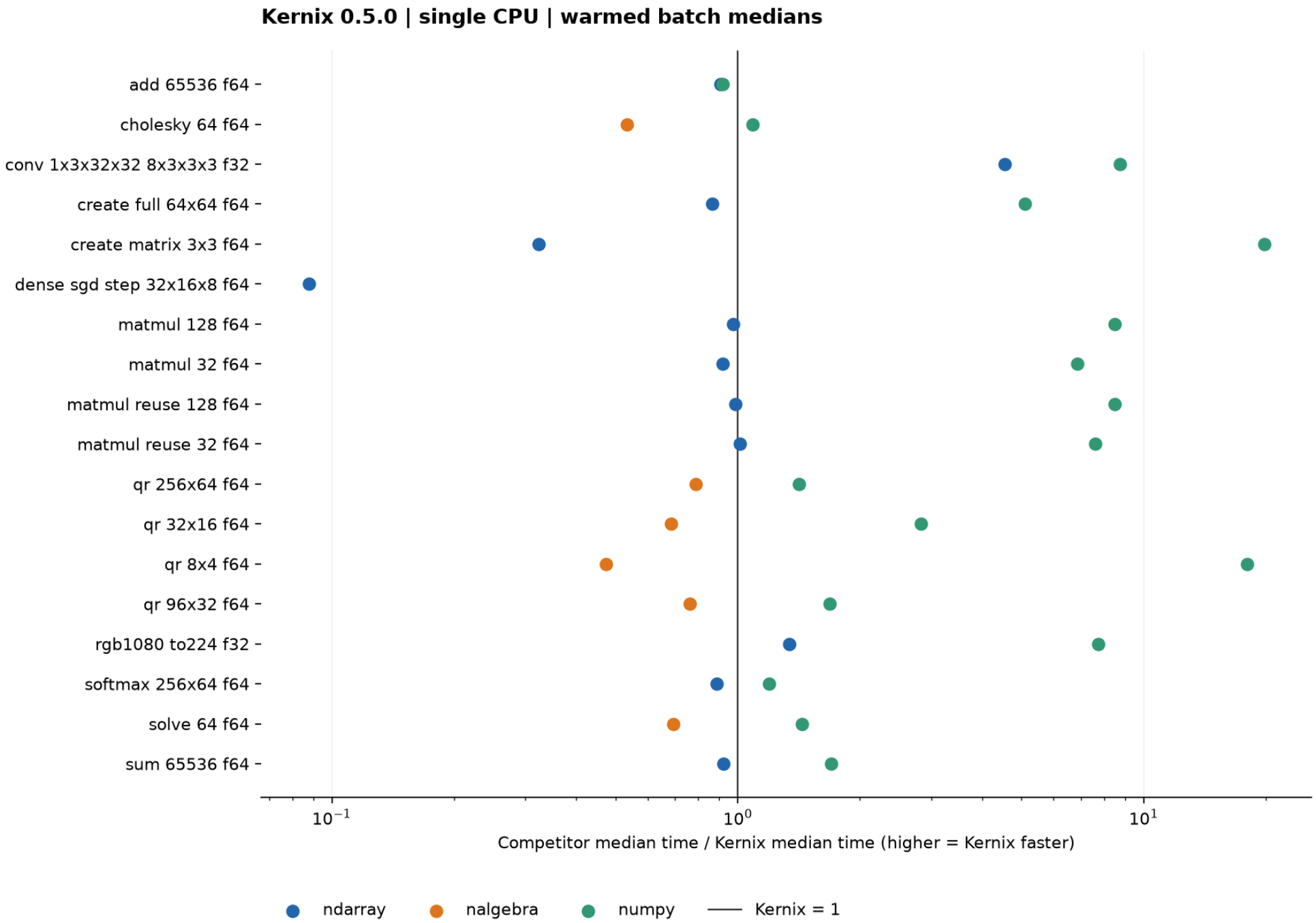


Figure 1: Median comparison; API and measurement limits apply.